MAJOR COURSE OUTLINE

1. *Introduction to Computer System*
2. *Analyzing computer systems architecture.*
3. *Computer Arithmetic and Operations*
4. *The design of the control units and CPU of a processor.*
5. The structure of computer instruction set
6. The organization of different bus systems and their characteristics in a computer system.
7. The importance, organization and management of computer memory system.
8. Low-level parallelism and its implementation in a processor

- *Final Examination* *60*
- *At least 1 progress test for feedback* *10*
- *At least 10 home works to be assessed by the teacher* *30*

Definition: Computer Architecture is a branch of computer science that integrates several fields of electrical engineering and computer science in developing computer system while a building architecture is a field of engineering that deals with building and structure. Below are roles of computer architecture.

THE ROLE OF COMPUTER ARCHITECTURE

1. The architect is responsible for the overall system design and performance.
2. Performance must be measured against quantifiable specifications.
3. The architect uses various performance measurement tools to determine the performance of system and subsystem. These tools often take the form of benchmark programs that are designed to measure a particular aspect of performance.
4. The architect is likely to become involved in low-level details of the computer design. In other words, the architect may wear many hats during system design.
5. The architect often uses formal description languages to convey details of the design to other members of the design team.
6. The architect strives for harmony and balance in system design.

INTRODUCTION TO COMPUTER

The computer as we know it today had its beginning with a 19th century English mathematics professor name Charles Babbage. He designed the Analytical Engine and it was this design that the basic framework of the computers of today are based on.

Generally speaking, computers can be classified into three generations. Each generation lasted for a certain period of time, and each gave us either a new and improved computer or an improvement to the existing computer.

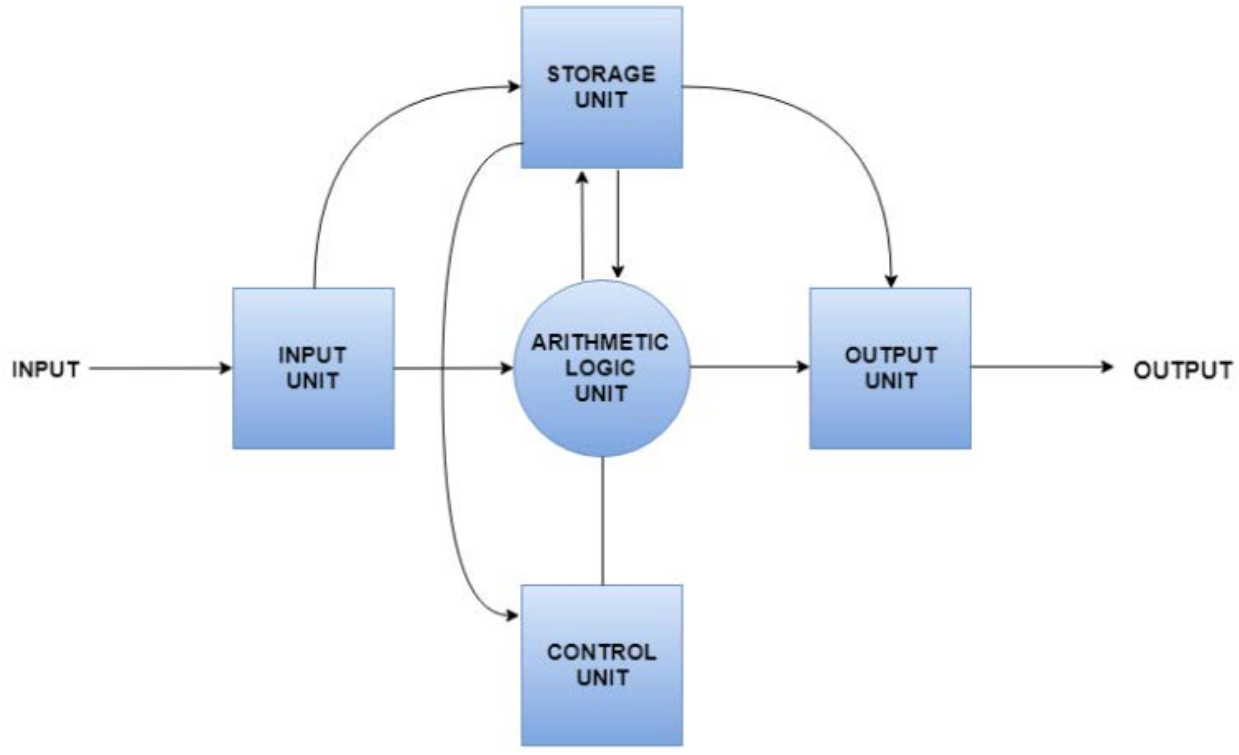
- First generation uses the **vacuum tubes** for processing. Computers of this generation could only perform single task, and they had no operating system.
- Second generation uses **transistors** instead of vacuum tubes which were more reliable.
- Third generation uses **integrated circuit**. With this invention computers became smaller, more powerful more reliable and they are able to run many different programs at the same time.

As a result of the various improvements to the development of the computer we have seen the computer being used in all areas of life. It is a very useful tool that will continue to experience new development as time passes.

ADVANTAGES OF COMPUTER IN SOCIETY

- Computer has made a very vital impact in the society. It has change the world into global village. The use of computer technology has affected every field of life.
- People are using computer to perform different tasks quickly. It takes less time to solve any problem.
- Many organizations are using computer to save the data and record of their customers. Banks also use computer to manage any financial transaction.
- Computer can also be used as educational tools. Students use internet for information. On internet different websites help student get information for their use.
- Computer is also used in medical researches, business, weather and all other institutions in the world.
- We can communicate to other people through computers by sending messages to other person through emails etc.

A computer system is basically a machine that simplifies complicated tasks. It should maximize performance and reduce costs as well as power consumption. The different components in the Computer System Architecture are Input Unit, Output Unit, Storage Unit, Arithmetic Logic Unit, Control Unit etc.
The diagram below shows the flow of data between these units.



FUNCTIONAL PART OF A COMPUTER

- CPU is the portion of a computer system that carries out the instructions of a computer program, and is the primary element carrying out the computer functions. The CPU in addition incorporates a number of registers that serve the purpose of providing the CPU itself with a small local (i.e. on the same chip) high speed memory. The central processing unit carries out each instruction of the program in sequence, to perform the basic arithmetic, logical, and input/output operations of the system. The CPU is also responsible for sequential fetching, decoding, and

executing the instruction. The CPU consists of several components include control unit and an ALU.

The input data travels from input unit to ALU. Similarly, the computed data travels from ALU to output unit. The data constantly moves from storage unit to ALU and back again. This is because stored data is computed on before being stored again. The control unit controls all the other units as well as their data.

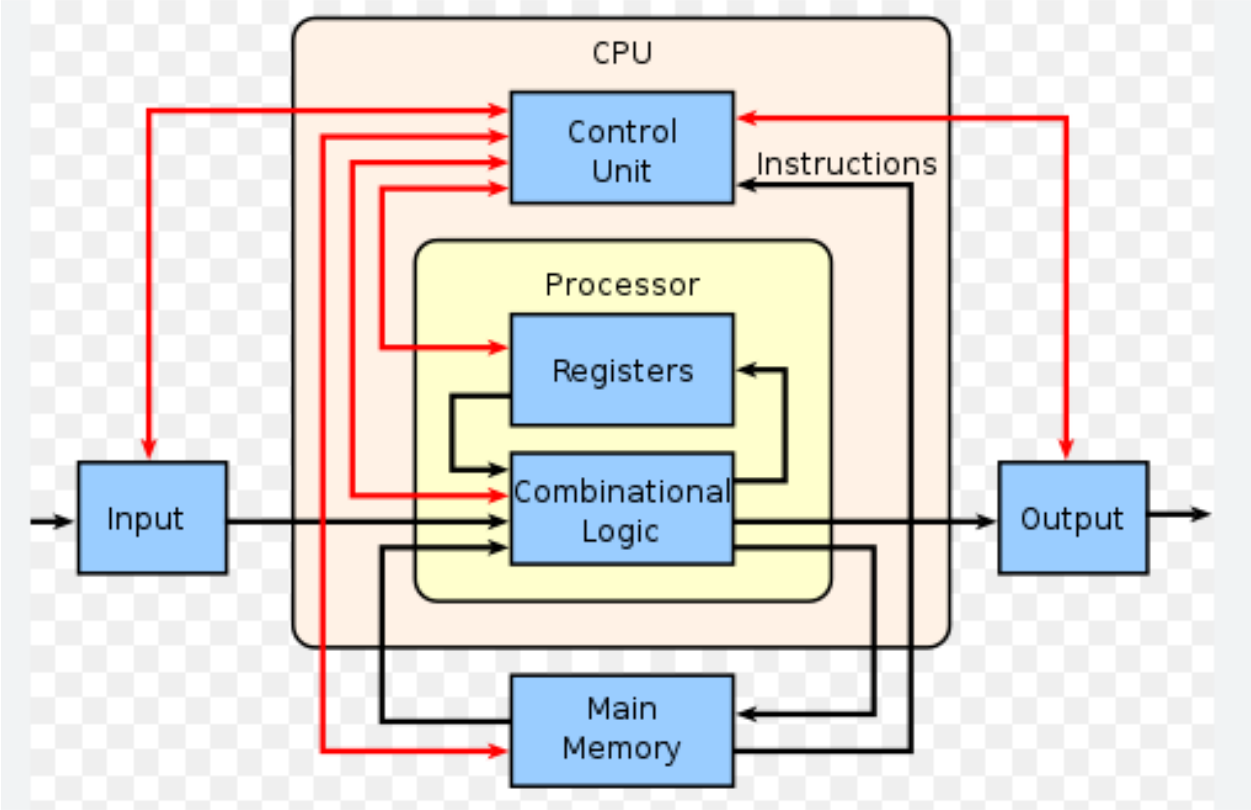
Details

- **Input Unit:** The input unit provides data to the computer system from the outside. It links the external environment with the computer. It takes data from the input devices, converts it into machine language and then loads it into the computer system. Keyboard, mouse etc. are the most commonly used input devices.
- **Output Unit:** The output unit provides the results of computer process to the users i.e it links the computer with the external environment. Most of the output data is the form of audio or video. The different output devices are monitors, printers, speakers, headphones etc.
- **Storage Unit:** Storage unit contains many computer components that are used to store data. It is traditionally divided into primary storage and secondary storage. Primary storage is also known as the main memory and is the memory directly accessible by the CPU. Secondary or external storage is not directly accessible by the CPU. The data from secondary storage needs to be brought into the primary storage before the CPU can use it. Secondary storage contains a large amount of data permanently.
- **Arithmetic Logic Unit:** All the calculations related to the computer system are performed by the arithmetic logic unit. It can perform operations like addition, subtraction, multiplication, division etc. The control unit transfers data from storage unit to arithmetic logic unit when calculations need to be performed. The arithmetic logic unit and the control unit together form the central processing unit.
- **Control Unit:** This unit controls all the other units of the computer system and so is known as its central nervous system. It transfers data throughout the computer as required including from storage unit to central processing unit and vice versa. The control unit also dictates how the memory, input output devices, arithmetic logic unit etc. should behave.
- Registers are high speed memory which is located inside of CPU and serves to keep temporary data. The ALU uses registers to hold data that is being processed.

JOHN VON NEUMANN ARCHITECTURE

The first computer systems developed were not able to store programs and program data was inputted using switches. In 1945 John Von Neumann was credited with designing the fundamental concept behind all modern computer systems. Von Neumann Architecture describes a system where a single control unit manages the Fetch-Execute cycle, and both instructions and data are stored in the same memory unit. His design included a *processing unit*, a *program counter*, *memory* to store data and instructions and external storage and input and output mechanisms. In the Von Neumann architecture:

- The program is stored in the main memory and instructions are fetched and executed sequentially. There is a single memory and bus system for accessing both data and programs.
- The diagram below shows the components of the central processing unit (CPU) and how they communicate with each other and with the other computer system components. Look closely at the components, their functions and the way in which program instructions are executed.
- The Central processing unit (CPU) is responsible for processing and executing program instructions, performing calculation and comparisons, as well as coordinating the behavior of other hardware. The CPU comprises of several different components as show in the diagram below.



Von Neumann Architecture

- The CPU lies at the heart of any Computer System, and is equivalent to the brain in a human. There are many processors in a modern computer. CPUs can contain one or more processing units known as cores.

It is common for computers to have two (dual), four (quad) or even more cores. CPUs with multiple cores have more power to run multiple programs at the same time.
A CPU is an integrated circuit that is responsible for performing arithmetic, logical and I/O operations. CPUs perform these operations as instructed by a computer program. A CPU is made up of three components:

- An Arithmetic & Logic Unit (ALU).
- A Control Unit (CU).
- Registers (Cache).

THE FETCH-EXECUTE CYCLE

For a program to be executed on a computer, it must be loaded into the computer's main memory. The processor then locates and accesses the program, which it then runs each instruction in turn. When the program is loaded, the processor is given a start address of where the program is stored in the main memory in order to access it. To run the program, the processor fetches an instruction, decodes it and then executes it. The program executes one instruction at a time and this is known as the **Fetch-Decode-Execute cycle.**

DEFINITION:

Fetch-Execute Cycle: This programmed instruction controls the computer state: Fetch Wait and Execute.

FETCH: The fetch state is used to read the instruction from the memory and to decode it. It goes through the following steps:

i. Contents of the memory location specified by the program counter are read into MBR.
ii. The content of PC is incremented by 1.
iii. Instruction is decoded, operation code of the instruction is transferred to Instruction Register (IR) to cause enact and the address port is transferred into MAR.
iv. Computer will then enter the execute state.

EXECUTE: The execute state is used in all memory reference instruction except jump.

i. For – instance – which need an operand from the memory the execute state is to read the operand into the MBR and to perform the operation specified by the opcode of the instruction, leaving the result in the accumulator. These instructions are ADD, LOAD, e.t.c.
ii. For the store instruction, this state is used to deposit the contents of the accumulator in the specified memory location.
iii. For the jump – to – sub-routine instructions, during this state, the contents of the PC are transferred into the PC in order to change the program control.
iv. **WAIT**: During this state, the computer is idle. It will only check whether to remain idle or go into a fetch.

MEMORY: Memory refers to the part of a computer system that stores data for use by the CPU. The data includes program files and data files used by programs. The main memory components are situated on computer chips, known as semi-conductors. A semi-conductor is a material that can conduct electricity under certain conditions, but not in others, making it useful for controlling electrical current. There are different types of memory that a computer system uses, they can be classified as volatile, Non-volatile and Virtual Memory.

- RAM is used to hold programs currently being executed, and the data the programs are using. When a program is to be executed, it has to be loaded from the hard disk into the main memory, so that the processor can access the instructions. Any data needed for that program to run is also loaded into main memory. The *processor cannot access secondary storage directly*. The main purpose of RAM is to act as a temporary storage for programs and data, while the program is being executed.

TYPES OF RAM:

- Static RAM (SRAM): Can hold data without being refreshed for as long as there is a power supply. More expensive than DRAM.
- Dynamic RAM (DRAM): Needs to be refreshed by frequently reading and rewriting the contents because its stored charge does not last very long. DRAM is more widely used because it is cheaper and takes up less space than SRAM.

BENEFITS OF HAVING MORE RAM

Computers with less RAM may run slowly due to having to make use of Virtual Memory.
Computers with more RAM can run more applications or more memory-intensive applications making the system run faster overall.
Upgrading RAM is relatively cheap and easy to do, replacing RAM with a higher capacity or higher speed, will improve the system performance.

- **READ ONLY MEMORY (ROM)**: ROM is a type of non-volatile memory that stores essential data such as a computer's configuration settings. Essential data is pre-installed onto ROM chips when a computer system is made. This type of memory is necessary to enable a computer to obtain instructions and information about the hardware from the moment it is switched on. Once a file has been stored on a ROM, it can be read but cannot be changed by the user. ROM can be accessed even if your computer has been switched off for months.

RAM Vs ROM

RAM	ROM
Volatile - Data is lost when power is turned off (Temporary Memory)	Non-volatile - Data is not lost when power is turned off (Permanent Memory)
Stores user data/programs/part operating system currently in use	Used to store BIOS/Bootstrap loader which is required at start-up
Memory can be written to or read from	Memory can only be read from, but not written to

INSTRUCTION SET

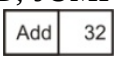
An instruction is a set of codes that the computer processor can understand. The code is usually in 1s and 0s, or machine language. It contains instructions that control the movement of bits and bytes within the processor.

Example of some instruction sets −

- **ADD** − Add two numbers together.
- **JUMP** − Jump to designated RAM address.
- **LOAD** − Load information from RAM to the CPU.

INSTRUCTION FORMAT AND ITS VARIOUS CONSTITUENTS

When the assembler processes an Instruction it converts the instruction from its mnemonics form to standard machine language format called the "Instruction format".

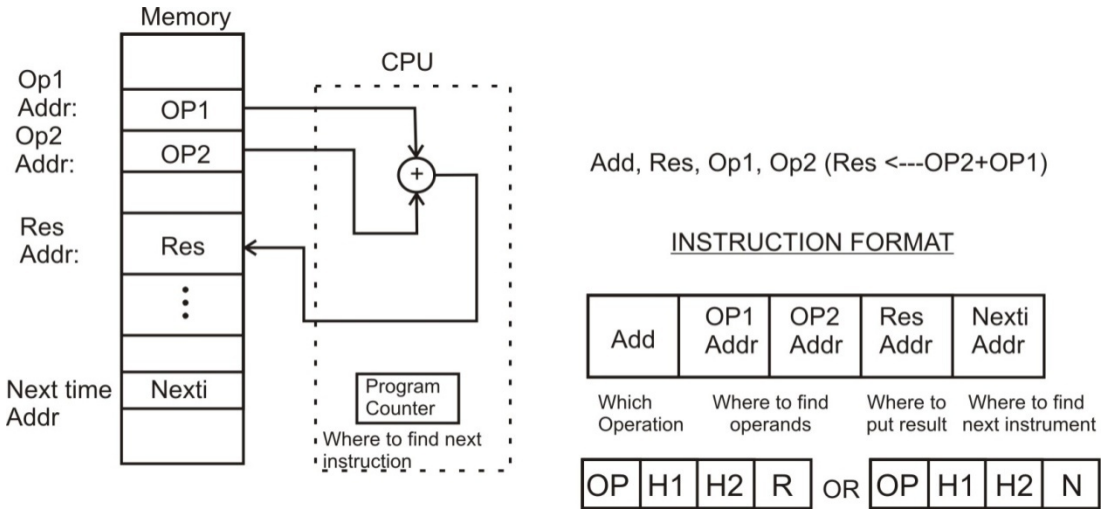
Each instruction consists of at least two parts – the operator and the operand. The operator specifies the specific operation to be performed by the processor e.g. ADD, JUMP, STOP, SHIFT etc. The operand specifies the data or address in memory to be operated upon - 

.

For instruction to be meaningful, it must have the following for two address instruction format:

i. The operation (OP) code e.g. Shift, stop, Add, e.t.c.
ii. The address of the first operand (H1).
iii. The address of second operand (H2).
iv. Address of the result (R)
v. Address of next instruction (N)
vi. Additional control information

A diagram below describes the sequence of operation and instruction format of three (3) address machine. The inclusion within the CPU of a program counter that always points to the next instruction eliminates the need to specify the address of the next instruction in all but the class of branch instructions. It is now the responsibility of the control unit to know the size of the currently executing instruction, so the control unit can advance the PC to point to the next address beyond. Machines with a program counter but no operand storage in the CPU are known as 3-Address machines. The figure below shows the operation of a 3 – Address machine and the corresponding instruction format. Note the inclusion of a program counter that points to the next instruction. It also shows that instructions and data may be stored in different parts of memory. The instruction format now includes only for fields one that specifies which operation is to be performed, two that specify operand memory address, and one that specifies the result address.



Example of how an instruction A=B-C get executed in digital system in detail

1. Clear the Memory Address Register (MAR): i.e. set the address of the first instruction.
2. Transfer 1st instruction into MBR.
3. Decode content MBR into R_0, R_1, R_2, R_3, R_4.
4. Address of 1st operand in MAR.
5. 1st Operand in MBR.
6. Transfer 1st operand into R.
7. Address of 2nd operand in MAR.
8. 2nd operand in MBR
9. Transfer 2nd operand to R_2.
10. Address of result in MAR.
11. Perform the operation specified by the operation code transfer the result into R_3 e.g. R_3<- R_1+R_2.
12. Load back the result into specified address in memory.
13. Address of next instruction in MAR.
14. Go back to step 2.

SOME DEFINITION

i. Latency is the time between the start of a process and its competition.
ii. Throughput is the amount of work done per unit time.
iii. Prefetching that is fetching the next instruction or instructions into an instruction queue before the current instruction is complete.

TYPES OF INSTRUCTION SET: Generally, there are two types of instruction set used in computers.

i. **Reduced Instruction set Computer (RISC)**: A number of computer designers recommended that computers use fewer instructions with simple constructs so that they can be executed much faster within the CPU without having to use memory as often. This type of computer is called a Reduced Instruction Set Computer. The concept of RISC involves an attempt to reduce execution time by simplifying the instruction set of computers.

Characteristics of RISC

- Relatively few instructions.
- Relatively few addressing modes.
- Memory access limited to load and store instructions.
- All operations done within the register of the CPU.
- Single-cycle instruction execution.
- Fixed length, easily decoded instruction format.

- Hardwired rather than micro programmed control.

A characteristic of RISC processors' ability is to execute one instruction per clock cycle. This is done by overlapping the fetch, decode and execute phases of two or three instructions by using a procedure referred as pipelining.

ii. **Complex Instruction Set Computer (CISC)**: CISC is a computer where a single instruction can perform numerous low-level operations like a load from memory and a store from memory, etc. The CISC attempts to minimize the number of instructions per program but at the cost of an increase in the number of cycles per instruction.

Characteristics of CISC

- A large number of instructions typically from 100 to 250 instructions.
- Some instructions that perform specialized tasks and are used infrequently.
- A large variety of addressing modes- typically from 5 to 20 different modes.
- Variable length instruction formats.
- Instructions that manipulate operands in memory.

The architecture of the Central Processing Unit (CPU) operates the capacity to function from "Instruction Set Architecture" to where it was designed. The architectural design of the CPU is Reduced instruction set computing (RISC) and Complex instruction set computing (CISC). CISC has the capacity to perform multi-step operations or addressing modes within one instruction set. It is the CPU design where one instruction works several low-level acts.

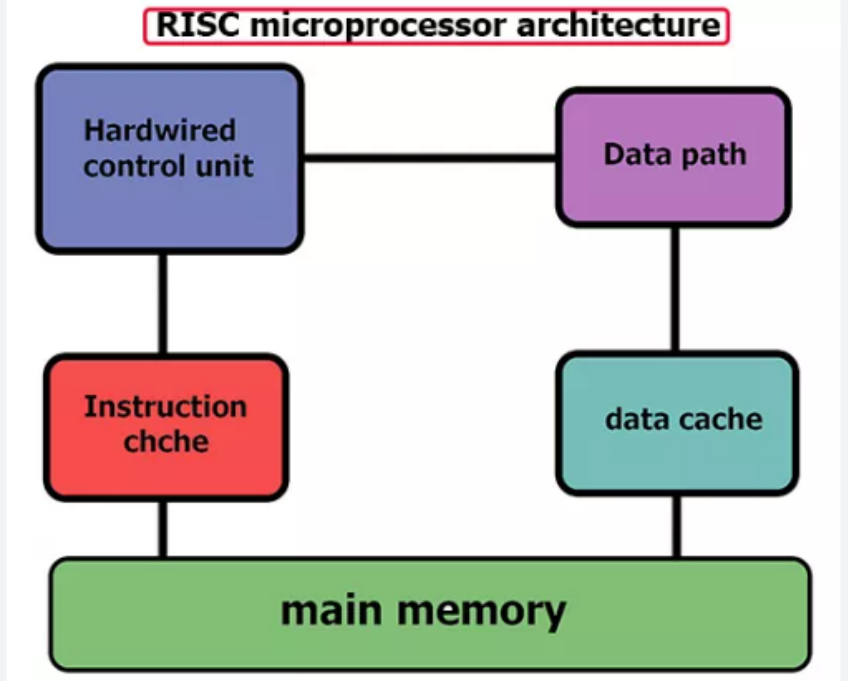
Reduced instruction set computing is a Central Processing Unit design strategy based on the vision that a basic instruction set gives great performance when combined with a microprocessor architecture. This architecture has the capacity to perform the instructions by using some microprocessor cycles per instruction.

Definition of ISA: An instruction set or instruction set architecture ISA) is a set of machine instruction that a computer is able to perform. Any operation not provided by the hardware via user and therefore not in the instruction set can be provided by using more than one instruction.

Characteristics of Instruction set architecture

1. The ISA is the interface between the software and hardware.
2. It is the set of instructions that bridges the gap between high level languages and the hardware.
3. For a processor to understand a command, it should be in binary and not in high level language. The ISA encodes these values.
4. The ISA also defines the items in the computer that are available to a programmer. For example, it defines data types, registers, addressing modes, memory organization etc.
5. Register are high addressing modes are the ways in which the instructions locate their operands.

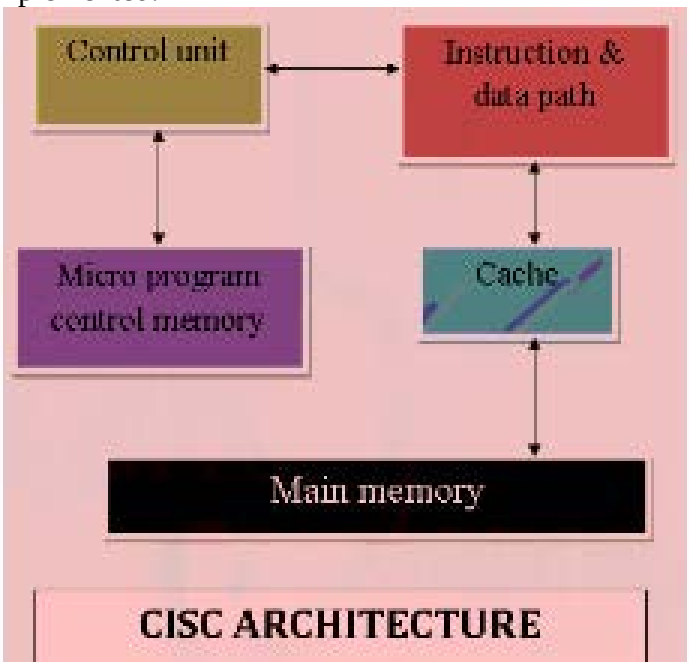
RISC ARCHITECTURE: This is a small set of instructions. Here, every instruction is expected to attain very small jobs. In this machine, the instruction sets are modest and simple, which help in comprising more complex commands. Each instruction is about a similar length; these are wound together to get compound tasks done in a single operation.



Reduced Instruction Set Computer is a microprocessor that is designed to carry out few instructions at a similar time. Based on small commands, these chips need fewer transistors, which make the transistors inexpensive to design and produce.

CISC Architecture

The Complex Instruction Set Computer is a CPU design plan based on single commands, which are skilled in executing multi-step operations. CISC computers have small programs. It has a huge number of compound instructions, which take a long time to perform. Here, a single set of instructions is protected in several steps; each instruction set has an additional than 300 separate instructions. Maximum instructions are finished in two to ten machine cycles. In CISC, instruction pipelining is not easily implemented.



- Instruction Set Architecture is a medium to permit communication between the programmer and the hardware. Data execution part, copying of data, deleting, or editing is the user commands used in the microprocessor, and with this microprocessor, the Instruction set architecture is operated.
- The main keywords used in the above Instruction Set Architecture are as below
    a. **Instruction Set:** Group of instructions given to execute the program and they direct the computer by manipulating the data. Instructions are in the form-Opcode (operational code) and Operand. Where, the opcode is the instruction applied to load and store data. The operand is a memory register where instruction is applied.
    b. **Addressing Modes:** Addressing modes are how the data is accessed. Depending upon the type of instruction applied, addressing modes are of various types such as a direct mode where straight data is accessed or indirect mode where the location of the data is accessed.

CPU performance is given by the fundamental law

$$CPU\ Time = \frac{Seconds}{Program} = \frac{Instructions}{Program} X \frac{Cycles}{Instructions} X \frac{Seconds}{Cycle}$$

Basic formula for calculating CPU time (in seconds)
CPU TIME = I* CPI* T

Where

I = Number of instructions in program
CPI = Average cycles per instructions
T = clock cycle time

Example, given

I = 30,
CPU time = 1hour,
CPI = 30sec.
Calculate the clock cycle time

Solution:

CPU TIME = I* CPI* T
T = CPU Time/I * CPI
1*60*60 = 30 * 30* T
T= 3600/900 = 4sec

Thus, CPU performance is dependent upon Instruction Count, CPI (Cycles per instruction), and Clock cycle time. And all three are affected by the instruction set architecture.

COMPARISON BETWEEN RISC AND CISC

The RISC processors have a smaller set of instructions with few addressing nodes. The CISC processors have a larger set of instructions with many addressing nodes.

DEFINITION OF PIPELINING AND ADVANTAGES

Pipelining is an implementation technique whereby multiple instructions are overlapped in execution.

ADVANTAGES OF PIPELINE

1. The cycle time of the processor is reduced, thus increasing instruction issue-rate in most cases.

2. Some combinational circuits such as adders or multipliers can be made faster by adding more circuitry. If pipelining is used instead, it can save circuitry as a more complex combinational circuit.
3. The ability to take new instructions at each clock cycle.
4. It increase the efficiency of each module within the CPU as compared to non- pipeline.

Pipelining as a modern technique used both in computer and digital electronic devices to increase instruction throughput with six (6) instructions to be executed.

A pipeline instruction means starting, or issuing the next instruction prior to the completion of the currently executing one. A pipeline is like an assembly line. In an automobile assembly line, there are many steps, each contributing something to the construction of the car. Each step operates in parallel with the other steps therefore increasing the total amount of work done in a unit time.

Six (6) instructions to be executed.

Time	**Execution**
0	Six instructions are waiting to be executed.
1	1. The green instruction is fetched from memory
2	1. The green instruction is decoded 2. The purple instruction is fetched from memory
3	1. The green instruction is executed (actual operation is performed) 2. The purple instruction is decoded. 3. The blue instruction is fetched.
4	1. The green instruction results are written are written back to the register file or memory. 2. The purple instruction is executed. 3. The blue instruction is decoded. 4. The red instruction is fetched.
5	1. The green instruction is completed 2. The purple instruction is written back. 3. The blue instruction is executed. 4. The red instruction is decoded 5. The white instruction is fetched
6	1. The purple instruction is completed. 2. The blue instruction is written back. 3. The red instruction is executed. 4. The white instruction is decoded 5. The pink instruction is fetched
7	1. The blue instruction is completed. 2. The red instruction is written back. 3. The white instruction is executed 4. The pink instruction is decoded
8	1. The red instruction is completed 2. The white instruction is write back 3. The pink instruction is executed
9	1. The white instruction is completed 2. The pink instruction is write back
10	1. The pink instruction is completed
11	1. All instructions are executed

PIPELINE HAZARD AND CLASSES

Pipeline hazard is a situation that prevents the next instruction in the instruction stream from being executing during its designated clock cycle. Hazards reduce the performance from the ideal speedup gained by pipelining. There are three classes of hazards:

1. Structural Hazards. They arise from resource conflicts when the hardware cannot support all possible combinations of instructions in simultaneous overlapped execution.
2 Data Hazards. They arise when an instruction depends on the result of a previous instruction in a way that is exposed by the overlapping of instructions in the pipeline.
3. Control Hazards. They arise from the pipelining of branches and other instructions that change the PC.

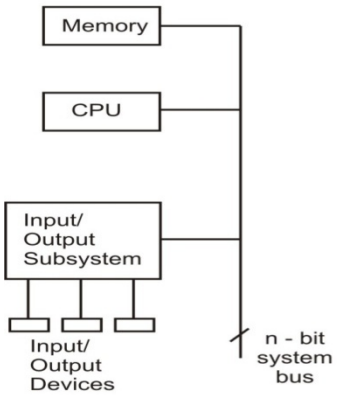
BUSES: The bus serves as a multiplexer allowing multiple devices to use it for communication. Because the bus is shared by multiple devices, it must provide both data path and a signaling path. Buses may be serial or parallel. Serial buses transmit information serially, one bit at a time. Parallel buses transmit a number of bits simultaneously.

Serial and Parallel Buses:

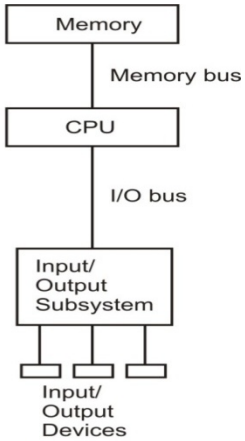
Buses that transfer several data bits at the same are called parallel buses. It is desirable to have wide buses because large amounts of data can be transferred quickly when multiple lines can be used. Parallel buses typically have 8, 16, 32 or 64 data lines.

Serial buses use the same line to transfer different data bits of the same byte/word.
Serial buses are less expensive than parallel buses, however, parallel buses have higher throughput.

Serial bus



Parallel bus



b) Two Buses

There are multiple registers and their functions which are as follows −

- **Load (LD)** − During the next clock pulse transition the information from the bus is transmitted to the register whose load (LD) input is enabled.
- **Memory Unit** − When the write input of the memory is activated, it holds the content of the bus. When the read input is activated, the memory places the 16-bit output onto the bus with the selection variables being S_2S_1 S_0 = 111..
- **Increment (INR) and Clear (CLR)** − When the INR signal is enabled, the contents of the specified register are incremented. The contents are cleared when the CLR signal is enabled.
- **Address Registers (AR)** − The address of the memory for the next read and write operation is determined. It receives or sends an address from or to the bus when selection inputs S_2S_1 S_0=001

is used and the load is enabled. With inputs INR and CLR, the address gets incremented or cleared.

- **Program Counter (PC)** − The address of the next instruction that is to be read from the memory is saved. It receives or sends an address from or to the bus when selection inputs S_2S_1 S_0 = 010is applied and the load input is enabled. With inputs INR and CLR, the address gets incremented or cleared.
- **Data Register (DR)** − The data register includes the data to be written into memory or data that is to be read from the memory. It receives or sends an address from or to the bus when selection inputs are S_2S_1 S_0 = 011 applied and the load input is enabled. With inputs INR and CLR, the address gets incremented or cleared.
- **Accumulator (AC)** − Accumulators are beneficial in executing the register micro-operations including complement, shift, etc. The results acquired are again sent to the accumulator. An accumulator stores the intermediate arithmetic and logic results.
- **Instruction Registers (IR)** − The IR stores the copy of the instruction that the processor has to implement. The instruction that is read from the memory is stored in the IR. It receives or sends instruction code from or to the bus when selection inputs S_2S_1 S_0 = 111 are applied and the load input is enabled.
- **Temporary Register (TR)** − The temporary storage for variables or results is supported by the temporary register. It receives or sends the temporary data from or to the bus when selection inputs S_2S_1 S_0 = 011 are applied and the load input is enabled. With inputs INR and CLR, the address gets incremented or cleared.
- **Input Registers (INPR)** − It includes 8 bits to hold the alphanumeric input information. Input device shifts its serial data into the 8-bit register. The data is moved to AC via the adder/logic circuit with load enabled.
- **Output Registers (OUTPR)** − The data is received from AC and moved to the output device.
- **Memory Buffer Register (MBR)**: This is used for reading the data out of memory or for writing the data into the memory.
- **Memory Address Register (MAR)**: This is the only means of addressing specific memory locations. It size depends on the number of words the computer can address directly. For example if it has 10 bit, then it will be able to address 2^{10} = 1024 = 1K words of main memory directly.
- **Instruction Register (IR)**: It is used to keep operation code of the instruction currently being performed by the machines. If the operation code has 3 bits, then the IR will have 3 bits. The instruction Register decides the operation codes and initiates the step necessary for the execution of the instruction.
- **Major State Register (MSR)**: All operations in the computer are performed step by step. One step represents the basic computer cycle and in modern machines, it lasts for about 1μS. These instructions are performed in a single cycle. The MSR determines the stage in the cycle that the processing is (fetch = 1, execute = 2, Wait = 0)
- **Go/Wait Register (GR)**: This is used to indicate to the computer when to go into action and when to be idle.

CONCEPT OF BUS ARBITRATION

A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time. Control signals determine which register is selected by the bus during each particular register transfer.

A program stays in the memory unit of the computer and has a sequence of instructions. The program is executed by going through a cycle for each instruction. Each instruction cycle is now subdivided into a sequence of sub cycles or phases. In the basic computer each instruction cycle has the following parts:

1. Fetch an instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory if the instruction has an indirect address.
4. Execute the instruction.

After the completion of step 4, the control goes back to step 1 to fetch, decode and execute the next instruction. This process continues indefinitely unless a HALT instruction is encountered.

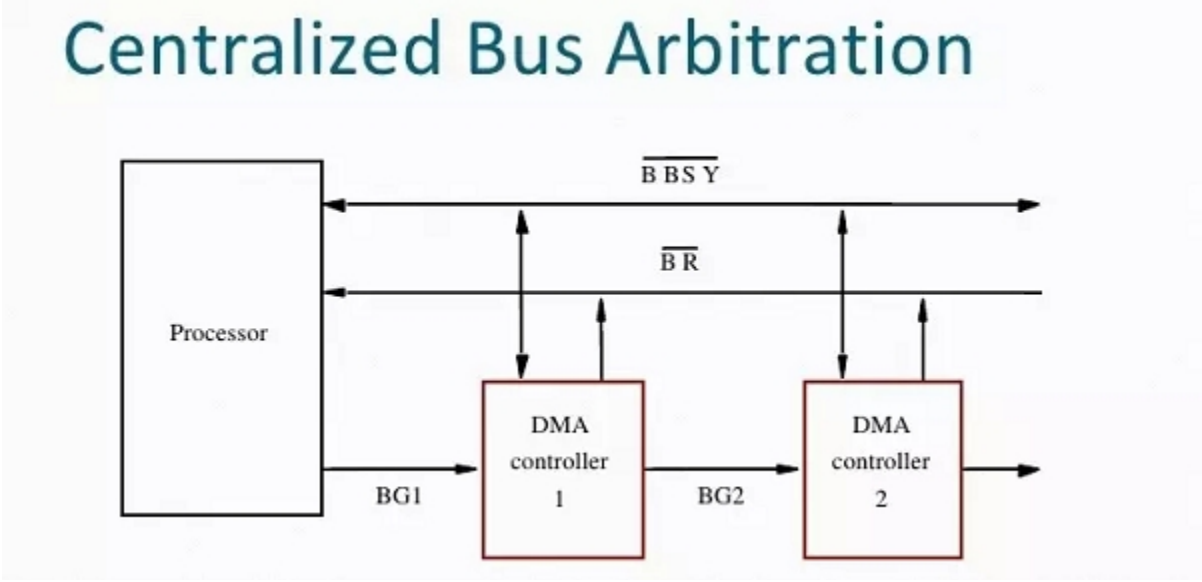
Definition: Bus Arbitration

- A device that initiates data transfers on the bus at any given time is called a bus master.
- In a computer system, there may be more than one bus master such as a Direct Memory Access (DMA) controller or a processor etc.
- These devices share the system bus and when a current master bus relinquishes another bus can acquire the control of the processor.
- Bus arbitration is a process by which next device becomes the bus controller by transferring bus mastership to another bus.

TYPES OF BUS ARBITRATION

There are two types of bus arbitration namely

1. Centralised Arbitration.
2. Distributed Arbitration.

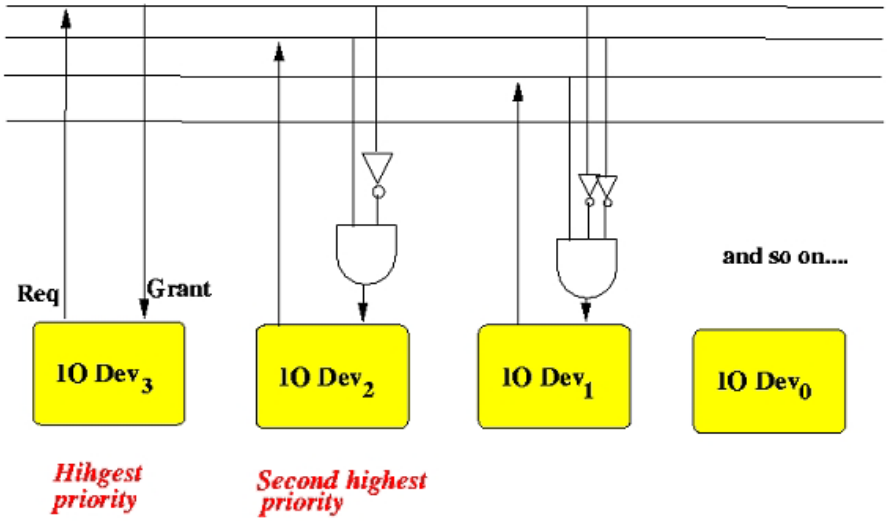


Only single bus arbiter performs the required arbitration and it can be either a processor or a separate DMS controller.

There are three arbitration schemes which run on centralized arbitration.

- **a) Daisy Chaining** − It is a simple and cheaper method where all the masters use the same line for making bus requests.
- **b) Polling Method** − In this method, the controller is used to generate address lines for the master. For example, if there are 8 masters connected in a system at least 3 address lines are required.
- **c) Independent Request** − In this scheme, each bus has its own bus request and a grant. The built-in priority decoder selects the highest priority requests and asserts the system.

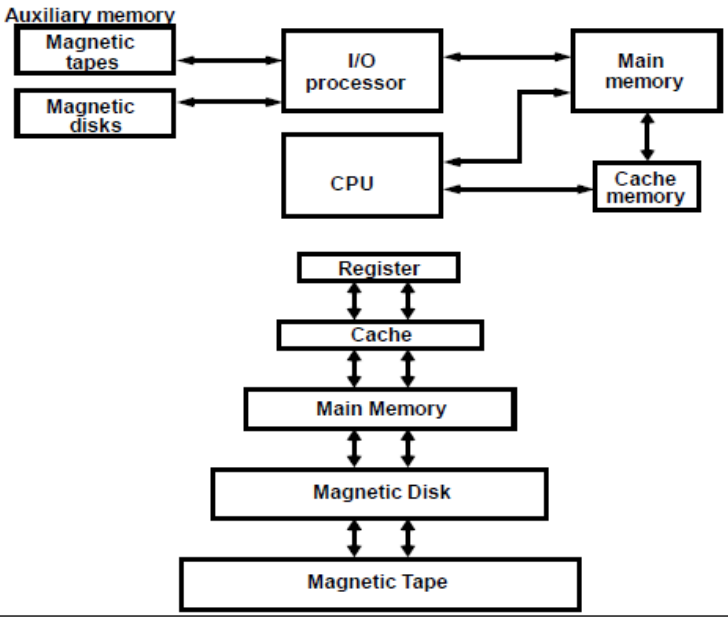
DISTRIBUTED ARBITRATION



- Here, all the devices participate in the selection of the next bus master.
- Each device on the bus is assigned a4 bit identification number.
- When one or more devices request control of the bus, they assert the start arbitration signal and place their 4-bit identification numbers on arbitration lines through ARB3.
- Each device compares the code and changes its bit position accordingly.
- It does so by placing a 0 at the input of their drive.
- The distributed arbitration is highly reliable because the bus operations are not dependent on devices.
- There are few basic guidelines or design elements that distribute to categorize and differentiate buses.

MEMORY HIERARCHY

Memory Hierarchy is to obtain the highest possible access speed while minimizing the total cost of the memory system



MEMORY ADDRESS MAP OF RAM AND ROM

Main Memory

The main memory is the central storage unit in a computer system.
Primary memory holds only those data and instructions on which computer is currently working.

- It has limited capacity and data is lost when power is switched off.
- It is generally made up of semiconductor device.
- These memories are not as fast as registers.
- The data and instruction required to be processed reside in main memory.
- It is divided into two subcategories RAM and ROM.

Memory address map of RAM and ROM

The designer of a computer system must calculate the amount of memory required for the particular application and assign it to either RAM or ROM.
The interconnection between memory and processor is then established from knowledge of the size of memory needed and the type of RAM and ROM chips available.
The addressing of memory can be established by means of a table that specifies the memory address assigned to each chip.
The table, called a **memory address map**, is a pictorial representation of assigned address space for each chip in the system.

Memory connections to CPU:

- RAM and ROM chips are connected to a CPU through the data and address buses
- The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs.

Auxiliary Memory:

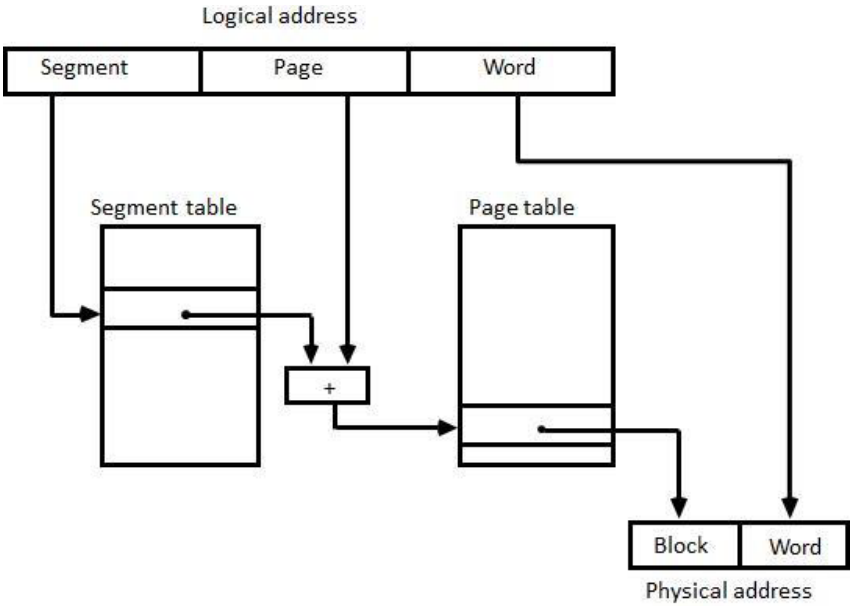
- **Magnetic Tape:** Magnetic tapes are used for large computers like mainframe computers where large volume of data is stored for a longer time. In PC also you can use tapes in the form of cassettes. The cost of storing data in tapes is inexpensive.
- **Magnetic Disk:** Magnetic disks used in computer are made on the same principle. It rotates with very high speed inside the computer drive. Data is stored on both the surface of the disk. Magnetic disks are most popular for direct access storage device. Each disk consists of a number of invisible concentric circles called tracks. Information is recorded on tracks of a disk surface in the form of tiny magnetic spots. For Example-Floppy Disk.
- **Sectors:** Tracks are further divided into sectors, which hold a block of data that is read or written at one time; for example, READ SECTOR 782, WRITE SECTOR 5448. In order to update the disk, one or more sectors are read into the computer, changed and written back to disk.
- **Optical Disk:** With every new application and software there is greater demand for memory capacity. It is the necessity to store large volume of data that has led to the development of optical disk storage medium. Optical disks can be divided into the following categories:

1. Compact Disk/ Read Only Memory (CD-ROM
2. Write Once, Read Many (WORM)
3. Erasable Optical Disk

LOGICAL TO PHYSICAL ADDRESS MAPPING

- ***Segment:*** A segment is a set of logically related instructions or data elements associated with a given name.
- ***Logical address:*** The address generated by segmented program is called a logical address.

- ***Segmented page mapping:*** The length of each segment is allowed to grow and contract according to the needs of the program being executed. Consider logical address shown in figure below.



The logical address is partitioned into three fields.

i. The segment field specifies a segment number.
ii. The page field specifies the page within the segment and
iii. word field gives specific word within the page.

A page field of k bits can specify up to 2k pages. A segment number may be associated with just one page or with as many as 2k pages.
Thus the length of a segment would vary according to the number of pages that are assigned to it.
The mapping of the logical address into a physical address is done by means of two tables, as shown in above. The segment number of the logical address specifies the address for the segment table.
The entry in the segment table is a pointer address for a page table base.
The page table base is added to the page number given in the logical address.
The sum produces a pointer address to an entry in the page table.
The concatenation of the block field with the word field produces the final physical mapped address.
The two mapping tables may be stored in two separate small memories or in main memory.
In either case, memory reference from the CPU will require three accesses to memory: one from the segment table, one from the page table and the third from main memory.
This would slow the system significantly when compared to a conventional system that requires only one reference to memory.

Fragmentation is a phenomenon in which storage space is used inefficiently, reducing capacity or performance and often both. The exact consequences of fragmentation depend on the specific system of storage allocation in use and the particular form of fragmentation. In many cases, fragmentation leads to storage space being "wasted", and in that case the term also refers to the wasted space itself.
Fragmentation occurs in a dynamic memory allocation system when many of the free blocks are too small to satisfy any request. External Fragmentation: External Fragmentation happens when a dynamic memory allocation algorithm allocates some memory and a small piece is left over that cannot be effectively used. If too much external fragmentation occurs, the amount of usable memory is drastically reduced. Total memory space exists to satisfy a request, but it is not contiguous Internal Fragmentation:

Internal fragmentation is the space wasted inside of allocated memory blocks because of restriction on the allowed sizes of allocated blocks.

CONCEPT OF PIPELINING

Pipelining is a technique wherein a sequential process is disintegrated into sub operations, with each sub process being executed in a special dedicated segment that operates concurrently with all other segments. A pipeline can be visualized as a collection of processing segments acting as a channel for binary information flow. Each segment performs partial processing dictated by the way the task is segregated. The result obtained from the computation in each segment is transferred to the next segment in the pipeline. The first result is obtained after the data have passed through all segments. It is characteristic of pipelines that several computations can be in progress in distinct segments at the same time. The overlapping of computation is made possible by associating a register with each segment in the pipeline. The registers provide isolation between each segment so that each can operate on distinct clock at the same time simultaneously.

For example, suppose that we want to perform the combined multiply and add operations with a stream of numbers –Ai+Bi+Ci for i = 1, 2, 3.....7

The sub operations performed in each segment of the pipeline are as follows:

R1 ¬ Ai R2 ¬ Bi Input Ai and Bi
R3 ¬ R1 *R2 R4 ¬ Ci Multiply and input Ci
R5 ¬ R3 + R4 Add Ci to product

Instruction pipeline

Pipeline processing appears both in the data stream and in the instruction stream. An instruction pipeline reads consecutive instructions from memory. While previous instructions are processed in other segments, an instruction pipeline read consecutive instruction from memory. This causes overlapping of fetch and execute face and performance of simultaneous operations. One possible digression associated with such a scheme is that an instruction may cause a branch out of sequence. When this happens the pipeline must be cleared and all the instructions that have been read from memory after the branch instruction must be discarded.

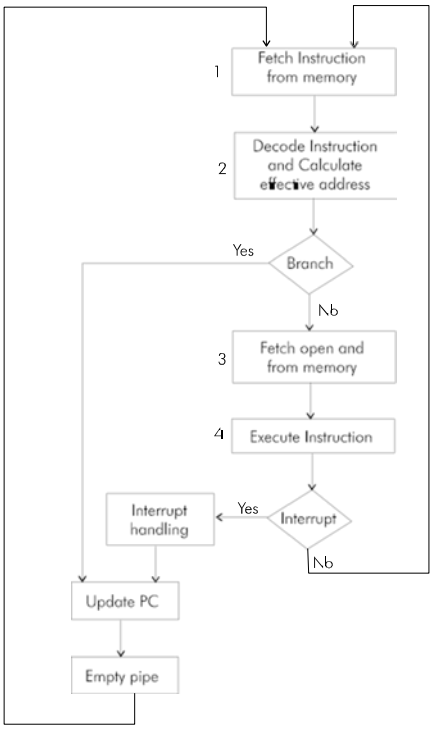
Take an example of a computer with an instruction fetch unit and an instruction execution unit designed to provide a two-segment pipeline. The instruction fetch segment can be implemented by means of a first-in, first-out (FIFO) buffer. This unit forms a queue rather than a stack. Whenever the memory is not used by execution unit, the control increments the program counter and uses its address value to read consecutive instructions from memory. The instructions are inserted into the FIFO buffer so that they can be executed on a first-in, first-out basis. Thus an instruction stream can be placed in a queue, waiting for decoding and processing by the execution segment. The instruction stream queuing mechanism is efficient for reducing the average access time to memory for reading instructions. Whenever there is space in the FIFO buffer, the control unit initiates the next instruction fetch phase. The buffer acts as a queue from which control then extracts the instructions for the execution unit.

In addition to the fetch an executive instruction, computers with complex instruction need other faces as well for complete processing of an instruction. A typical computer requires the following sequence of steps for instruction processing:

1. Fetch the instruction from memory
2. Decode the instruction.
3. Calculate the effective address.
4. Fetch the operands from memory.
5. Execute the instruction.
6. Store the result in the proper place

Four-Segment Instruction Pipeline

Assume that the decoding of the instruction can be combined with the calculation of the effective address into one segment. Assume further that most of the instructions place the result into a processor registers so that the instruction execution and storing of the result can be combined into one segment. This reduces the instruction pipeline into four segments. Figure below shows how the instruction cycle in the CPU can be processed with a four-segment pipeline. While an instruction is being executed in segment 4, the next instruction in sequence is busy fetching an operand from memory in segment 3. The effective address may be calculated in a separate arithmetic circuit for the third instruction, and whenever the memory is available, the fourth and all subsequent instructions can be fetched and placed in an instruction FIFO. Thus up to four sub operations in the instruction cycle can overlap and up to four different instructions can be in progress of being processed at the same time.



Four-Segment Instruction Pipeline

COMPUTER SYSTEMS ARCHITECTURE [COM 314], 1ST & 3RD SEMESTER, 2023/2024 ACADEMIC SESSION

Assignment One

I. Describe Brief Historical Background of computer system
II. Describe Architectural development and Style
III. Technological Development and Performance Measures
IV. Explain the below functional units in a computer system and their operations:
   a. Input/ Output units
   b. Arithmetic and Logic Unit
   c. Control Unit
   d. Memory Unit
   e. Registers

Assignment Two

I. Describe Basic processor architecture
II. Explain Fetch and execute cycle.
III. Explain Types of Computer architecture:
   a. Von Neumann's Architecture
   b. Reduced Instruction Set Computers (RISC) Architecture
   c. Complex instruction Set Computers (CISC) Architecture
IV. Explain RISC Design Principle with its Merit and Performance of RISC Architecture

Interpret concepts Number System
Interpret Integer Arithmetic
Describe two's Complement Representation
Describe two's complement Arithmetic
Explain Floating Point Arithmetic

Assignment Three

I. Define Control Unit
II. Describe the structure of control unit.
III. Explain Hardwired control unit
IV. Explain the functions of a control unit.
V. Differentiate types of control units
VI. Explain the design of Micro-programmed control unit.
VII. Describe CPU Basics components
VIII. Identify Register set
IX. Identify different components of Data path

Assignment Four

I. Explain CPU Instruction cycle
II. Define Instruction Set
III. design of computer instruction set
IV. List types of instruction set
V. The operation of an instruction set
VI. The instruction set of a typical computer system

Assignment Five

I. Describe Memory Location and Operation

II. Explain Addressing, immediate, Direct, Indirect, Indexed modes
   a. performance Measure
   b. Instruction Types:
   c. Data Movement Instruction
   d. Arithmetic and logical Instruction
   e. Sequencing instruction
   f. Input Output Instruction

Assignment Six

I. Explain the Bus concept
   a. Draw different bus architecture.
   b. Single bus and multiple bus architecture.
   c. Compare and contrast different bus architecture
   d. Synchronous and Asynchronous Buses
   e. Different Bus Arbitration
   f. Explain the organization of: ISA, EISA, VESA, PCI, USB, IDE, standard interface Bus systems.

Assignment Seven

I. Explain the concepts of Memory Hierarchy
   a. Memory structure of a computer system.
   b. Backing store, internal store
   c. Cache Memory and layers

II. The purpose and function of different level of memory in the overall structure

III. Explain Cache Memory Organization:
   a. Direct mapping
   b. Full associative Mapping
   c. Set Associative Mapping

Assignment Eight

I. Explain the concept of Main Memory and Virtual Memory.

II. Memory management technique:
   a. Page
   b. Segment
   c. Paged Segment

III. Explain the concept of parallel computing and how it can be achieved

IV. States the benefits of parallel computing

V. Explain Concept of Pipelining
   a. Basic pipeline for a typical computer system.
   b. Problems associated with pipeline operation
   c. Performance optimization using pipelining.